

ALGORITMOS E ESTRUTURAS DE DADOS**Atividade:** Lista, Fila e Pilha**Professor:** Thiago Goveia**1. Verdadeiro ou falso. Justifique as falsas.**

- A. () Em vetores, o acesso aleatório aos elementos é rápido, enquanto em listas encadeadas, pode ser mais lento.
- B. () Listas encadeadas são mais eficientes que vetores na inserção e remoção de elementos no meio da estrutura.
- C. () Uma classe abstrata contém a implementação dos métodos definidos nas classes “concretas”
- D. () Pilhas usam a abordagem LIFO (Last In, First Out), onde o último elemento inserido é o primeiro a ser removido.
- E. () Vetores alocam memória dinamicamente à medida que novos elementos são inseridos.
- F. () As listas encadeadas consomem mais memória em comparação com vetores, devido aos ponteiros de ligação entre os elementos.
- G. () Um Tipo Abstrato de Dados é definido por um conjunto de dados e operações que podem ser executadas sobre estes dados
- H. () As pilhas são úteis para a implementação de algoritmos recursivos.
- I. () Vetores permitem a inserção rápida de elementos em qualquer posição da estrutura.
- J. () Listas encadeadas facilitam a manipulação de grandes quantidades de dados, mas são menos eficientes para acessos aleatórios.
- K. () As filas seguem a política FIFO (First In, First Out), onde o primeiro elemento inserido é o primeiro a ser removido.

2. Preencha com vetor, lista, fila ou pilha (e listar, enfileirar, empilhar, etc). Justifique sua escolha.

- A. Em jogos de videogame, _____ são frequentemente utilizad@s para representar e gerenciar dados como as posições dos personagens, as coordenadas do mapa e a contagem de itens, permitindo o rápido acesso aos elementos e facilitando a movimentação e interações no ambiente virtual.
- B. Em aplicações de redes sociais, _____ são empregad@s para estruturar os feeds de postagens de usuários, gerenciando conexões e relacionamentos entre os perfis. Isso permite a rápida inserção de novas postagens, suportando um fluxo dinâmico de conteúdo.
- C. Nos sistemas de atendimento hospitalar, _____ são usad@s para organizar os pacientes que chegam. Esta estrutura permite um atendimento justo e eficiente, garantindo que os pacientes sejam atendidos em ordem cronológica, reduzindo o tempo de espera.
- D. Os navegadores web usam _____ para implementar o histórico de navegação. Cada página visitada é _____, permitindo a navegação de volta (utilizando o botão "Voltar").
- E. Na análise de dados e estatísticas, _____ são fundamentais para armazenar conjuntos de informações numéricas, facilitando cálculos matemáticos, representando dados em gráficos e possibilitando a realização de análises comparativas.
- F. Em sistemas operacionais e linguagens de programação, _____ são empregad@s para gerenciar a alocação de memória dinâmica, mantendo o controle de espaços livres e ocupados, viabilizando a alocação e liberação eficiente de memória.
- G. Sistemas de impressão utilizam _____ para organizar os trabalhos de impressão. Os

documentos enviados são _____, evitando conflitos e congestionamentos na impressora.

- H. Editores de texto e softwares de edição frequentemente implementam _____ para oferecer a funcionalidade de "Desfazer" (undo) e "Refazer" (redo), permitindo ao usuário voltar e avançar nas ações executadas durante a edição de textos.
- I. Em aplicativos de geolocalização e mapas interativos, _____ são usados para representar coordenadas geográficas.
- J. _____ são empregad@s em sistemas de gerenciamento de tarefas para organizar cronogramas e agendas, permitindo a adição dinâmica de novas tarefas e o acompanhamento sequencial das atividades a serem realizadas.

3. Mostre graficamente o estado da memória (ponteiros) após cada operação a seguir:

- A. insere("A"), insere("B"), insere("C"), insere("D"), retira("B"), retira("E");
- B. enfileira("A"), enfileira("B"), enfileira("C"), desenfileira(), desenfileira(), enfileira("D");
- C. empilha("A"), empilha("B"), empilha("C"), desempilha(), desempilha(), empilha("D");

```
public class Lista {
    private Object item[];
    private int primeiro, ultimo, pos;

    public Lista(int maxTam) {
        this.item = new Object[maxTam];
        this.pos = -1;
        this.primeiro = 0;
        this.ultimo = this.primeiro;
    }

    public void insere (Object x) throws Exception {
        if(this.ultimo >= this.item.length){
            throw new Exception ("Erro : A lista está cheia" );
        }else{
            this.item[ this.ultimo] = x;
            this.ultimo = this.ultimo + 1;
        }
    }

    public Object retira(Object chave) throws Exception {
        if (this.vazia() || chave == null)
            throw new Exception("Erro: A lista está vazia");

        int p = 0;
        while (p < this.ultimo && !this.item[p].equals(chave))
            p++;

        if (p >= this.ultimo) return null;

        Object item = this.item[p];
        this.ultimo = this.ultimo - 1;

        for (int aux = p; aux < this.ultimo; aux++)
            this.item[aux] = this.item[aux + 1];

        return item;
    }

    public boolean vazia() {
        return (this.primeiro == this.ultimo);
    }
}
```

```
private static class Celula {
    Object item;
    Celula prox;
}

public class Fila {
    public Fila() { // Cria uma Fila vazia
        this.frente = new Celula();
        this.tras = this.frente;
        this.frente.prox = null;
    }

    public void enqueue(Object x) {
        this.tras.prox = new Celula();
        this.tras = this.tras.prox;
        this.tras.item = x;
        this.tras.prox = null;
    }

    public Object dequeue() throws Exception {
        Object item = null;
        if (this.vazia())
            throw new Exception("Fila vazia");
        this.frente = this.frente.prox;
        item = this.frente.item;
        return item;
    }

    public boolean vazia() {
        return (this.frente == this.tras);
    }
}
```

```
class Pilha {
    private Celula topo;
    private int tam;

    public Pilha() {
        this.topo = null;
        this.tam = 0;
    }

    void empilha(Object x) {
        Celula aux = this.topo;
        this.topo = new Celula();
        this.topo.item = x;
        this.topo.prox = aux;
        this.tam++;
    }

    public Object desempilha() throws Exception {
        if (this.vazia()) {
            throw new Exception("Pilha vazia");
        }
        Object item = this.topo.item;
        this.topo = this.topo.prox;
        this.tam--;
        return item;
    }

    public boolean vazia() {
        return (this.topo == null);
    }
}
```